



Fast scalable construction of ([compressed] static | minimal perfect hash) functions [☆]



Marco Genuzio ^{a,1}, Giuseppe Ottaviano ^b, Sebastiano Vigna ^{a,*,1}

^a Dipartimento di Informatica, Università degli Studi di Milano, Milan, Italy

^b Facebook, Menlo Park, USA

ARTICLE INFO

Article history:

Received 3 October 2018

Received in revised form 1 July 2019

Accepted 9 September 2019

Available online 15 January 2020

Keywords:

Minimal perfect hash functions

Static functions

Compressed functions

Succinct data structures

ABSTRACT

Recent advances in the analysis of random linear systems on finite fields have paved the way for the construction of constant-time data structures representing static functions and minimal perfect hash functions using less space with respect to existing techniques. The main obstacle for any practical application of these results is the time required to solve such linear systems: despite they can be made very small, the computation is still too slow to be feasible.

In this paper, we describe in detail a number of heuristics and programming techniques to speed up the solution of these systems by orders of magnitude, making the overall construction competitive with the standard and widely used MWHC technique, which is based on hypergraph peeling. In particular, we introduce *broadword programming* techniques for fast equation manipulation and a *lazy Gaussian elimination* algorithm. We also describe a number of technical improvements to the data structure which further reduce space usage and improve lookup speed.

Our implementation of these techniques yields a minimal perfect hash function data structure occupying 2.24 bits per element, compared to 2.68 for MWHC-based ones, and a static function data structure which reduces the multiplicative overhead from 1.23 to 1.024. For functions whose output has low entropy, we are able to implement feasibly for the first time the Hreinnsson–Krøyer–Pagh approach, which makes it possible, for example, to store a function with an output of 10^6 values distributed following a power law of exponent 2 in just 2.76 bits per key instead of 20.

© 2020 Elsevier Inc. All rights reserved.

1. Introduction

Static functions are data structures designed to store arbitrary functions from finite sets to integers; that is, given a set of n keys S , and for each $x \in S$ a value v_x , with $0 \leq v_x < 2^b$, a static function will retrieve v_x given $x \in S$ in constant time. Closely related are *minimal perfect hash functions* (MPHFs), where only the set S is given, and the data structure yields an injective numbering of S into the first n natural numbers.

While these tasks can be easily implemented using hash tables, static functions and MPHFs are allowed to return *any* value if the queried key is not in the original set S ; this relaxation enables to break the information-theoretical lower bound

[☆] This paper is an extended version of [1] and [2].

* Corresponding author.

¹ Sebastiano Vigna and Marco Genuzio are supported by a Google Focused Grant.

of storing the set S . Indeed, constructions for static functions achieve just $O(nb)$ bits of space and MPHFs $O(n)$ bits of space, regardless of the size of the keys. This property makes static functions and MPHFs powerful techniques when handling, for instance, large sets of strings, and they are important building blocks of space-efficient data structures such as (compressed) full-text indexes [3], monotone MPHFs [4,5], Bloom filter-like data structures [6], and prefix-search data structures [7]. There exist also a few theoretical proposals for the construction of *compressed static functions*, in the sense that the space used per key should be close to the empirical entropy of the sequence of values [8,9,6].

An important line of research, both theoretical and practical, involves lowering the multiplicative constants in the big- O space bounds while keeping feasible construction times. In particular, compressed static functions have never been implemented due to the unfeasible construction time, albeit they might provide in principle faster access due to the smaller memory footprint.

In this paper, we build on recent advances in random linear systems theory and in perfect hash data structures [10,11], to achieve practical static functions with the lowest space bounds so far, and construction time comparable with widely used techniques. The new results, however, require solving linear systems rather than a simple depth-first visit of a hypergraph, as it happens in current state-of-the-art solutions. In the non-compressed case, or in the case of minimal perfect hashing, we have one equation per key; in the case of compressed static function, we follow the approach of Hreinnsson, Krøye and Pagh [9], which yields one equation *per bit of output*, making the problem even harder.

Since we aim at structures that can manage billions of keys, the main challenge in making such structures usable is taming the cubic running time of Gaussian elimination at construction time. To this purpose, besides exploiting a combination of engineering techniques typical of the field, we introduce novel techniques based on *broadword programming* [12] and a lazy version of *structured Gaussian elimination*. We obtain data structures that are significantly smaller than widely used hypergraph-based constructions while maintaining or improving the lookup times and providing still feasible construction time. For compressed functions, we show experimentally that the increase in construction time is acceptable in the case of low entropy.

All implementations discussed in this paper are distributed as free software as part of the Sux4J project [13].

2. Notation and tools

We use von Neumann's definition and notation for natural numbers, identifying n with $\{0, 1, \dots, n-1\}$, so $2 = \{0, 1\}$ and 2^b is the set of b -bit numbers.

Our model of computation is a unit-cost word RAM with word size w . We assume that the universe U from which keys are extracted has size $O(2^{cw})$ for some constant c , so that its elements can be hashed in constant time.

A k -hypergraph on a vertex set V is a subset E of $\binom{V}{k}$, the set of subsets of V of cardinality k . An element of E is called a *hyperedge*. The *degree* of a vertex is the number of hyperedges which contain it. An *induced subgraph* is given by a subset of vertices and all edges incident on those vertices (and no other vertices). The t -core of a hypergraph is its maximal induced subgraph having degree at least t .

A hypergraph is *peelable* if it is possible to sort its hyperedges in a list so that for each hyperedge there is a vertex that does not appear in the following elements of the list (such sorting can be easily found in linear time using a depth-first visit). A hypergraph is peelable if and only if it has an empty 2-core. It is *orientable* if it is possible to associate with each hyperedge a distinct vertex. Clearly, a peelable hypergraph is orientable, but the converse is not necessarily true.

If we fix a field $\mathbf{F}$ and assign to each hyperedge e of a k -hypergraph a value $v_e \in \mathbf{F}$, we can associate with the hypergraph a linear system in which variables are vertices and hyperedges represent equations: more precisely, a hyperedge $e \subseteq V$ represents the equation $\sum_{v \in e} v = v_e$. This correspondence induces a bijection between linear systems with $|E|$ equations, each containing k variables with coefficient one, and k -hypergraphs with values assigned to hyperedges. We will switch frequently between the two views.

The *empirical 0-th order entropy* $\mathcal{H}_0$ of a sequence $v_0, v_1, \dots, v_{n-1} \in \Sigma$ is defined as

$$\mathcal{H}_0 = \sum_{\sigma \in \Sigma} -p_\sigma \log p_\sigma,$$

where p_σ is the frequency of σ in the sequence $v_0, v_1, \dots, v_{n-1}$.

Finally, we recall the definition of the k -XORSAT problem. A k -XORSAT instance is given by n clauses of exactly k boolean literals (i.e., variables or negated variables): given an assignment for the variables, the value of a clause is just the XOR of the value of its variables. A k -XORSAT instance is *satisfiable* if there is a variable assignment such that all clauses are true. Recent results [14,15] have shown that for each k there exists a constant δ_k , with $\delta_k \rightarrow 1$ as $k \rightarrow \infty$, such that if there are more than $\delta_k n$ variables then a random instance is satisfiable with high probability.

Given an $\mathbf{F}_2$ -linear system of n equations, each containing exactly k variables, we can easily construct an equivalent k -XORSAT instance: we add one clause per equation containing the only k variables in the equation, possibly negating one variable if the constant term is zero. At that point, the solvability of the system is equivalent to the satisfiability of the k -XORSAT instance.

3. The problems

Let us first state our problems more formally.

Problem 1 (*Static functions*). We are given a set of keys S , $|S| = n$, out of a universe U , and a function $f : S \rightarrow 2^b$ associating a b -bit value with each key. Our problem is to store information so that given $x \in U$ we can compute in constant time a value that will be exactly $f(x)$ if $x \in S$.

The key point here is that we give no condition when $x \notin S$ (except that the computation must still be performed in constant time), so the information-theoretical lower bound for storing S as a subset of U does not apply. However, storing the set of values requires in general nb bits.

Problem 2 (*Minimal perfect hash functions*). We are given again a set of keys S , and we want to store a bijective function $f : S \rightarrow n$ under the same conditions of Problem 1.

The function is not specified, so the lower bound on the set of values does not apply, either. Indeed, the lower bound per key is very small (about ≈ 1.44 bits [16]), and independent from S . Some theoretical constructions reach the lower bound, but they work only for preposterously large n [17].

Problem 3 (*Static compressed functions*). Consider an arbitrary set Σ of possible output values. We are given a function $f : S \rightarrow \Sigma$; denoting with $\mathcal{H}_0$ the empirical entropy of the sequence of the output values $f(x_0), f(x_1), \dots, f(x_{n-1})$, we want to store again the function f with the same constraints of Problem 1, but using space per key close to $\mathcal{H}_0$.

The constant-time constraint sometimes is relaxed to *expected* constant time, or to constant time with high probability. Construction of such structures should be performed in (expected) linear time.

4. Background and related work

4.1. Random hypergraphs: the MWHC approach

In their seminal paper [18], Majewski, Wormald, Havas, and Czech (MWHC hereinafter) introduced the first compact construction for static functions using the connection between linear systems and hypergraphs. To store a function $f : S \rightarrow 2^b$ with b -bit values, they generate a random system with $n = |S|$ equations in m variables of the form

$$w_{h_0(x)} \oplus w_{h_1(x)} \oplus \dots \oplus w_{h_{k-1}(x)} = f(x) \quad x \in S. \quad (1)$$

Here $h_i : U \rightarrow m$ are k fully random hash functions, and the w_i 's are variables representing b -bit vectors. Due to bounds on the acyclicity of random hypergraphs, if the ratio between the number of variables and the number of equations is above a certain threshold γ_k , the system can be almost always triangulated in linear time by peeling the associated hypergraph, and thus solved; in other words, we have both a probabilistic guarantee that the system is solvable, and that the solution can be found in linear time.

The data structure is then a solution of the system (i.e., the values of the variables w_i): storing the solution makes it clearly possible to compute $f(x)$ in constant time. The space usage is approximately $\gamma_k b$ bits per key. The constant γ_k depends on the degree k , and attains its minimum at $k = 3$ ($\gamma_3 \approx 1.23$).

We remark that the original paper [18] does not use b -bit vectors: rather, the codomain of f is an arbitrary integer μ , and Equation (1) is interpreted using standard addition modulo μ . In practical applications, there is no advantage in using values of μ that are not powers of 2, and modulo operations are very slow, so the $\oplus$ -based approach is preferred. However, as we will see in the next section, in the case of minimal perfect hash functions it is useful to perform computations on $\mathbf{F}_3$.

4.2. Minimal perfect hash functions

Chazelle, Kilian, Rubinfeld and Tal [19], unaware of the MWHC construction, proposed it independently, but also exploited the fact that a peelable hypergraph is also orientable: since as a side effect of the peeling process each hyperedge can be assigned a unique vertex (or, equivalently, each equation can be assigned a unique variable), each key can be assigned injectively an integer in $\lceil \gamma_k n \rceil$. Then, we just need to modify Equation (1) so that instead of storing $f(x)$, we store which of the k vertices of the hyperedge is the assigned one, and this can be done in approximately $\gamma_k \lceil \log k \rceil$ bits per key. At retrieval time, the value we store makes it possible to recover the unique integer assigned to the key.

To make the perfect hash function minimal, that is, a function to n , rather than $\lceil \gamma_k n \rceil$, it is possible to add a *ranking* data structure. A bit vector of size $\lceil \gamma_k n \rceil$ records which vertices have been assigned to a hyperedge, and then with additional $o(n)$ bits it is possible to compute in constant time the number of ones preceding a given one (i.e., its rank; see [20] for practical implementations).

Again, the best k is 3, which yields theoretically a $2.46n + o(n)$ bits data structure [21], using 2 bits per variable, provided that the system is actually solved on $\mathbb{F}_3$: in this way, the value 3 never appears in the solution, and it can be used instead of zero to mark vertices associated with hyperedges. In this way, vertices assigned to hyperedges have always nonzero values and are the only ones with nonzero values because of the way the solution is computed by the peeling process. In the end, it is easy to adapt ranking structures so as to rank nonzero pairs of bits, rather than ones, eliminating the need for the additional bit vector (see [21] for details).

4.3. HEM

Botelho, Pagh and Ziviani [21] introduced a practical external-memory algorithm called Heuristic External Memory (HEM) to construct MPHFs for sets that are too large to store their hypergraph in memory. They replace each key with a *signature* of $\Theta(\log n)$ bits computed with a random hash function and check that no collision occurs. The signatures are then sorted and divided into small buckets using the most significant bits, and a separate representation of the function f for the keys in each bucket is computed using the approach described above. The representations of all such functions are then concatenated and stored into a single global bit array, taking care of storing separately *offsets*, that is, the starting position of the representation of each function. While devised for MPHFs, HEM can be made to work with any construction technique based on hypergraphs and linear systems.

4.4. Cache-oblivious constructions

As an alternative to HEM, in [22] the authors propose *cache-oblivious* algorithms that use only scanning and sorting to peel hypergraphs and compute the corresponding structures. The main advantage is that of avoiding the cost of accessing the offset array of HEM without sacrificing scalability.

4.5. CHD

Specifically for the purpose of computing MPHFs Belazzougui, Botelho, and Dietzfelbinger [23] introduced a completely different construction, called CHD (compressed hash-and-displace), which, at the price of increasing the expected construction time makes it possible, in theory, to reach the information-theoretical lower bound of ≈ 1.44 bits per key.

4.6. Fingerprinting

Recently, Müller *et al.* [24] introduced a completely new technique for minimal perfect hashing. A series of bit arrays of decreasing size, called *levels*, is used to record information about collisions between keys. More precisely, all positions in the first level to which a single key is mapped by a random hash function are marked with a one. Then, one takes all keys which participated in collisions, and try to map them into the second level, using the same strategy, and so on. As a result, one obtains a perfect hashing of the key set: to retrieve the output associated with a key, one just maps the key in turn to each level until a one is found, and then the (overall) position of the one yields a distinct number for each key. A constant-time ranking structure over the concatenation of the bit arrays is then used, as in the case of hypergraph-based techniques, to turn the perfect hash function into a minimal perfect hash function.

Fingerprint-based minimal perfect hash functions have the advantage of being extremely tunable for speed (both for construction and lookup) by setting suitably the length of each level, and thus the space usage: at 3 bits per elements, for example, the authors report that only 1.56 levels are accessed on average, resulting in a very low number of cache misses. However, the best space result for space is 2.58 bits per key, which is not competitive, for example, with CHD or the techniques described in this paper. Moreover, at that size lookup time is very slow.

4.7. Beyond hypergraphs

The MWHC construction for static functions can be improved: Dietzfelbinger and Pagh [10] note that it is possible to make the constant in front of the nb space bound for static functions *arbitrarily small* using bounds on full-rank random matrices on $\mathbb{F}_2$: by Calkin's theorem [25], a constant β_k exists such that if there are more than $\beta_k n$ variables then the matrix associated with the linear system has full rank with high probability. Contrarily to γ_k , which has a finite minimum, β_k vanishes quickly as k increases, thus the denser the rows, the closer m can be to n . For example, if $k = 3$, $\beta_3 \approx 1.12 < \gamma_3 \approx 1.23$.

Unlike MWHC's linear-time peeling algorithm, general matrix inversion requires superquadratic time ($O(n^3)$ with Gaussian elimination); to obtain a linear-time algorithm, they shard the set S into small sets using a hash function, and compute the static functions on each subset independently; the actual construction is rather involved, to account for some corner cases (note that the HEM algorithm described above is essentially a practical simplified version of this scheme).

The main practical obstacle to this approach, before the results described in this paper, was that construction time is two orders of magnitude slower than that of the MWHC construction [26], making the whole approach unusable in practice. In Rink's Ph.D. thesis [11] these considerations are described in some detail.

Dietzfelbinger and Pagh show also that Calkin's bound implies that the hypergraph associated with the system is orientable, thus making it possible to construct perfect hash functions, too, in the style of Section 4.2.

4.8. Compression

The next natural step is some sort of *output-dependent compression*. A trivial approach is that of building a minimal perfect hash function $p : S \rightarrow n$, a Huffman code for the output values, concatenate the codewords representing $f(x)$ in the order induced by $p(x)$ and store the codeword boundaries using a succinct data structure. In this case, the space used for the concatenated Huffman code is close to the empirical entropy, but the ancillary data structures cause a constant overhead which becomes very significant when entropy is small (which is the interesting case). Moreover, it is necessary to store the mapping from the codewords to the outputs. Subsequent theoretical works have worked around some of these limitations [8,9,6].

We now recall the main ideas behind the construction described by Hreinnsson, Krøye, and Pagh in [9], which we chose as a basis of our engineering effort because of its simplicity. Let $c : \Sigma \rightarrow 2^*$ be a binary prefix-free encoding of Σ . We fix an integer k (in our implementations, $k = 3, 4$). The core of the construction is the creation of a bit array A and of k hash functions $h_{(i)} : U \rightarrow |A|$ with the following property: given a key $x \in S$, if we XOR the content of A starting at positions $h_0(x), h_1(x), \dots, h_{k-1}(x)$ (possibly wrapping around) we obtain a stream of bits which starts with $c(f(x))$. More precisely, the $|c(f(x))|$ bits given by

$$\bigoplus_{0 \leq i < k} A_{(h_i(x)+j) \bmod |A|}$$

for $0 \leq j < |c(f(x))|$ are exactly $c(f(x))$. Assuming that the longest codeword in the prefix-free code is a constant multiple of the machine word, the sequence of bits can be reconstructed and decoded in constant time. This assumption can be made to hold at the expense of space occupation by limiting the depth of the tree associated with the prefix-free code.

Which constraints does the condition above put on the bit array A ? It is easy to see that for every $x \in X$ the i -th bit of $c(f(x))$ gives rise to the equation

$$A_{(h_0(x)+i) \bmod |A|} \oplus A_{(h_1(x)+i) \bmod |A|} \oplus \dots \oplus A_{(h_{k-1}(x)+i) \bmod |A|} = c(f(x))_i.$$

Thus, we obtain a system of $\sum_{x \in X} |c(f(x))|$ equations on $\mathbf{F}_2$, each with exactly k variables. The authors apply once again Calkin's bound: A needs to be $\beta_k \sum_{x \in X} |c(f(x))|$ bits long, where β_k is the constant discussed in Section 4.7.

If the distribution associated with the prefix-free code matches exactly the distribution of the values, $\sum_{x \in X} |c(f(x))|/n$ is exactly $\mathcal{H}_0$, which suggests that in general the space used per key will be very close to $\beta_k \mathcal{H}_0$, that is, very close to the entropy. In practice, as we will see, there will be some ancillary data which must be stored, too, and we will not be able to work with a truly optimal code, but both limitations have a relatively small impact on the whole data structure.²

Note that in this process we shifted from n equations (for the non-compressed case) to $\sum_{x \in X} |c(f(x))|$ equations—an $\mathcal{H}_0$ -fold increase in the size of the linear system, to which an even larger increase in construction time follows, as Gaussian elimination is cubic.

5. New constructions

In this paper, we combine a number of new results and techniques to provide improved constructions or constructions that previously have not even been attempted in practice, due to their high computational costs. In this section we lay out our approach in general, and in the following sections we provide details—in particular, engineering details, which are essential to reach our goal.

First of all, we apply *offline bucketing*, a new variant of the HEM construction described in Section 4.3; however, in the compressed case we also gather statistics about the output values. In the end, we have reduced the problem to relatively small buckets (typically, a few thousand keys on average) of settable average size B .

Then, only in the compressed case we create a suitable prefix-free code (with constant decoding time) for the outputs. The code is necessary to carry out the construction described in Section 4.8. We adapt the standard decoding algorithm for canonical Huffman codes [27] to our case, in which we can read an entire codeword from memory at once, obtaining a very simple and fast decoding algorithm (a loop with a single exit test).

In all cases, we now generate the linear systems whose solutions will provide the representation of the function. We combine a number of recent results about k -XORSAT thresholds [14,15], their relationship to orientability and cuckoo hashing [28–30], and the extension of such thresholds to $\mathbf{F}_3$ [31], to argue that we can always reliably use the k -XORSAT

² The authors of [9] show that for distributions in which the most frequent element $\bar{\sigma}$ has a frequency very close to one, it is useful to store the set of keys mapped to $\bar{\sigma}$ using a separate approximate dictionary. The trick can be applied to our engineered construction orthogonally, so we will not discuss this issue further.

thresholds as a ratio vertices/hyperedges to obtain systems that are almost surely solvable. These thresholds are a strict improvement over those used previously, which were derived from peelability thresholds or from Calkin's theorem. We then solve the linear systems using new heuristics and programming techniques that improve the speed of Gaussian elimination of our random systems by orders of magnitude.

Finally, we store the data structure. While doing so, we exploit a few more engineering ideas to further reduce the final space usage.

6. Static functions

We start by discussing in detail the phases that concern the construction of static functions. Then, we describe the modifications that are necessary to handle minimal perfect hash function and compressed static functions.

6.1. Offline bucketing

In the first pass, given an expected bucket size B , we scan the input data and we assign each key to a different bucket using a random hash function. The purpose of this phase is to reduce the construction to buckets of average size B .

While this task might be easily performed in memory in linear time, we will perform this task offline—hence the name *offline bucketing*. The basic strategy we use is inspired by HEM [21], but we use a hashing scheme that makes it possible to actually obtain expected size B using a standard lexicographical sort.³

First, we replace the original pairs $\langle x, f(x) \rangle$, $x \in S$, with a simpler set of pairs $\langle s(x), f(x) \rangle$ where $s(x)$ is a digital fixed-length signature, and we save such pairs on disk. The signature length should be chosen so as to make unlikely a signature collision.⁴ The pairs $\langle s(x), f(x) \rangle$ are then sorted by $s(x)$, and if collisions are found the construction is repeated with a different signature. Thus, at the end of the first pass, we have on disk a sorted list of pairs $\langle s(x), f(x) \rangle$.

The final bucket assignment is generated using *fixed-point multiplication*: we interpret the value $u(x)$ of the upper t bits (e.g., $t = 64$) of $s(x)$ as a real number $\alpha(x) = u(x)/2^t$ in the interval $[0..1)$ represented in fixed-point arithmetic, and we assign to x the bucket $\lfloor \alpha(x) \cdot \lfloor n/B + 1 \rfloor \rfloor$. If we interpret $\alpha(x)$ as a real random uniform value in the unit interval, this operation corresponds to an *inversion* [33] returning a random uniform discrete value in $\lfloor n/B + 1 \rfloor$, which is exactly what we need.⁵

Since we use fixed-point arithmetic, this simply amounts to computing $\lfloor u(x) \cdot \lfloor n/B + 1 \rfloor / 2^t \rfloor$, which can be performed with a multiplication and a shift. Multiplication by a positive number and floor are monotone nondecreasing functions, so each bucket is made of contiguous keys in sorted order, and we can emit the buckets, one at a time, just by scanning the sorted keys.

We remark that as long as the amount of work done at construction time on a bucket is polynomially bounded in the bucket size, the resulting overall work will be linear in expectation. This happens because bucket sizes follow a binomial distribution with parameters $p = B/n$ and n , and the d -th moment of a random variable S with such binomial distribution can be bounded as follows [35]:

$$\mathbb{E}[S^d] = \sum_{i=0}^d \binom{d}{i} i! \binom{n}{i} \left(\frac{B}{n}\right)^i = \sum_{i=0}^d \binom{d}{i} \frac{n!}{n^i (n-i)!} B^i \leq \sum_{i=0}^d \binom{d}{i} B^i.$$

From a practical viewpoint, to speed up the sorting, the pairs $\langle x, f(x) \rangle$ are first divided into 256 on-disk *disk segments*, depending on the most significant bits of their hash value. The disk segments are then loaded in memory and sorted individually. The fact that we load in memory each segment makes it possible to use *exponential search* [36] to locate the start of each bucket.

6.2. Generating systems

We now analyze the buckets generated by the offline bucketing procedure, with the idea of generating a random linear system *independently for each bucket*. We will need for each bucket a *local seed* which will be used as a parameter in the system generation, so as to be able to generate different systems (by trying local seeds $0, 1, 2, \dots$) until we find a solvable one (which must happen soon, as we will generate systems that are asymptotically solvable with high probability).

Note that here the fact that we are using pairs $\langle s(x), f(x) \rangle$ provides a major practical speed improvement, since we have simply to combine $s(x)$ with the local seed and shuffle the resulting fixed-length bit vector to obtain a new, random-looking hash: there is no need to rehash the original keys.⁶

³ The hash function suggested for HEM by the authors can only guarantee that the average bucket size will be within a factor of two from B .

⁴ We use the 128 bits returned by the “short” SpookyHash [32] of the keys.

⁵ Indeed, fixed-point inversion has been since the early days the technique of choice for turning the bits returned by a pseudorandom number generator into a uniform discrete value in a finite range (see, e.g., [34]).

⁶ In practice, we initialize the 256-bit internal state of the short SpookyHash algorithm using the 128-bit signature and the local seed, and then perform a round of mixing.

The generation of the random linear system happens identically to the MWHC technique described in Section 4.1, but we use for the ratio vertices/hyperedges the threshold δ_k for k -XORSAT satisfiability [14,15]. The resulting system will be solvable with high probability as long as we consider b constant (recall that we are storing a function $f : S \rightarrow 2^b$).

From a theoretical viewpoint, the satisfiability of k -XORSAT instances and the solvability of linear systems on $\mathbf{F}_2$ are equivalent, but in our case variables are b -bit vectors. If $b > 1$ we can, however, slice the system for each bit: the equivalence will give us b k -XORSAT instances, all of which are solvable with high probability. Our system will be solvable when all b slices are solvable, which will happen with a lower probability than that of a single instance: still, for fixed b the system will be asymptotically solvable with high probability.⁷

Note that, as in the case of Calkin's threshold β_k , δ_k is a constant that tends to 1 as k grows ($\delta_3 \approx 1.089$, $\delta_4 \approx 1.024$). This is an obvious improvement over techniques based on hypergraph or full-rank matrices, as there are solvable systems whose matrices do not have full rank. For example, $\delta_3 \approx 1.089 < \beta_3 \approx 1.12 < \gamma_3 \approx 1.23$.

In the end, concatenating the bit arrays containing the solution of each bucket into a global bit array we use exactly the same space we would have used without offline bucketing. We have however to store the prefix sums of the number of variables in each bucket, so as to be able to compute the offset associated with a bucket, and the local seed of each bucket, to be able to generate the equation associated with a key.

6.3. Solving systems

We now have to solve (in principle by Gaussian elimination) a sequence of linear systems, one for each bucket, with s equations and $\delta_k s$ variables, where s is the bucket size (on average, B).

Peeling. As a preparatory step, we try to peel the hypergraph as in the classical MWHC construction. In our case the hypergraph will not be peelable: nonetheless, for small k a significant fraction of edges can be peeled, and the associated equations can be removed from the system, reducing the complexity of the subsequent phase.

In particular, we implement a new peeling algorithm that uses a single stack: the typical peeling algorithm performs a depth-first (or breadth-first) visit of the graph, following only hyperedges incident on nodes of degree one. As hyperedges are peeled, they are accumulated on a stack together with their *hinge*—the node of degree one from which they were peeled.

To reduce significantly memory usage, we exploit the *XOR trick* [22]: for each node, we keep track of the incident edges by storing the XOR of their indices. As we peel edges, we update the XOR, which means that at the moment in which we find a node of degree one, its incidence list contains exactly one edge, which can be retrieved directly. In particular, this means that we need just to stack hinges.

Starting from this observation, we developed an even leaner approach: since hinges are stacked in the same order they would be in a queue backing a breadth-first visit, we can simulate the queue simply with a pointer on the hinge stack, saving an array of the same size (the one backing the breadth-first visit) or the space used by the recursion stack (for a depth-first visit).

Broadword programming. Our first step towards a practical solution by Gaussian elimination is *broadword programming* [12] (a.k.a. SWAR—“SIMD in A Register”), a set of techniques to process simultaneously multiple values by packing them into machine words of w bits and performing the computations on the whole words.

In the case of static functions this is easy, as the inner loop of the Gaussian elimination is entirely composed of row operations: given vectors x and y , and a scalar α , compute $x + \alpha y$. It is trivial to perform this operation w elements at a time when the field is $\mathbf{F}_2$: we can just pack one element per bit, and since the scalar can be only 1 the sum is just a bitwise XOR. This approach provides an $O(w)$ speedup with respect to bit-by-bit computations, and it is extremely competitive with a sparse representation on systems with a small number of variables.

Finally, when performing back substitution we will need to compute row-matrix multiplications, where a row is given by the coefficients of an equation and the matrix contains the solutions computed so far. This can be achieved by iterating on the ones of the row and adding up the corresponding b -bit rows in the right-hand matrix. Such iteration can be performed by finding the LSB of the current row word, and deleting it with the standard broadword trick $x = x \& (x - 1)$.

Lazy Gaussian elimination. Even if armed with broadword algorithms, solving by Gaussian elimination systems of the size of a bucket (thousands of equations and variables) would be prohibitively slow, making the construction of our data structures an order of magnitude slower than the standard MWHC technique.

Structured Gaussian elimination aims at reducing the number of operations in the solution of a linear system by trying to isolate a number of variables appearing in a large number of equations, and then rewrite the rest of the system using just those variables. It is a heuristics developed in the context of computations of discrete logarithms, which require the solution of large sparse systems [37,38]. The standard formulation requires the selection of a fraction (chosen arbitrarily) of variables that appear in a large number of equations, and then a number of loosely defined refinement steps.

⁷ Experimentally, we see an evident increase in the percentage of unsolvable systems as b goes through very small values, but the percentage stabilizes around $b = 16$, with an increase of less than 50% from the case $b = 1$ in the case we examined experimentally. It is an interesting open problem to attempt an exact computation of the probability of a system generated in this case to be solvable.

We describe here a new parameterless version of structured Gauss elimination, which we call *lazy Gaussian elimination*. This heuristics turned out to be extremely effective on our systems, reducing the size of the system to be solved by standard elimination to around 4% of the original one.

Consider a system of equations on some field. At any time a variable can be *active*, *idle*, or *solved* and an equation can be *sparse* or *dense*. Initially, all equations are sparse and all variables are idle. We will modify the system maintaining the following invariants:

- dense equations do not contain idle variables;
- an equation can contain at most one solved variable;
- a solved variable appears in exactly one dense equation.

Our purpose is to modify the system so that all equations are dense, trying to minimize the number of active variables (or, equivalently, maximize the number of solved variables). At that point, values for the active variables can be computed by standard Gaussian elimination on the dense equations that do not contain solved variables, and solved variables can be computed easily from the values assigned to active variables.

The *weight* of a variable is the number of sparse equations in which it appears. The *priority* of a sparse equation is the number of idle variables in the equation. Lazy Gaussian elimination keeps equations in a min-priority queue, and performs the following actions:

1. If there is a sparse equation of priority zero that contains some variables, it is made dense. If there are no variables, the equation is either an identity, in which case it is discarded, or it is impossible, in which case the system is unsolvable and the procedure stops.
2. If there is a sparse equation of priority one, the only idle variable in the equation becomes solved, and the equation becomes dense. The equation is then used to eliminate the solved variable from all other equations.
3. Otherwise, the idle variable appearing in the largest number of sparse equations becomes active.

Note that if the system is solvable the procedure always completes—in the worst case, by making all idle variables active (and thus all equations dense).

Two observations are in order:

- The weight of an idle variable never changes, as in step 2 we eliminate the solved variable and modify the coefficients of active variables only. This means that we can simply sort initially (e.g., by countsor) the variables by the number of equations in which they appear, and pick idle variables in that order at step 3.
- We do not actually need a priority queue for equations: simply, when an equation becomes of priority zero or one, it is moved to the left or right side, respectively, of a deque that we check in the first step.

Thus, the only operations requiring superlinear time are the eliminations performed in step 2, and the final Gaussian elimination on the dense equations, which we compute, however, using broadword programming.

6.4. Data-structure representation

Finally, we write the data structure. Besides obvious metadata (the number of keys, etc.) it comprises the concatenated solutions of all linear systems, the offset of each bucket (more precisely, the sum of the number of variables appearing in previous buckets), and the bucket local seeds. This is the totality of the overhead imposed by offline bucketing with respect to constructing the function over the whole input set at once.

Instead of storing these two numbers separately, we combine them into a single 64-bit integer. The main observation that allows us to do so is that due to the extremely high probability of finding a good seed for each bucket, few random bits are necessary to store it: we can just use the same sequence of seeds for each bucket, and store the number of failed attempts before the successful one. In our experiments, this number is distributed geometrically and never greater than 24. If we are storing n keys, $64 - \lceil \delta_k \log n \rceil$ bits are available for the seed, which are more than sufficient for any realistic n .

7. Minimal perfect hash functions

We now discuss the modification of the strategy we just discussed for the case of minimal perfect hash functions. Note that in this case, the offline bucketing step uses only the signatures $s(x)$.

7.1. Generating systems

When generating the random linear systems, we exploit the orientability thresholds in [28–30], which are shown to be the same as those of k -XORSAT solvability; for example, when a random 3-hypergraph has a vertices/hyperedges ratio larger than δ_3 , it contains a nonempty 2-core, but the hypergraph is orientable and we can obtain an orientation using


```

uint64_t add_mod3_step2(uint64_t x, uint64_t y) {
    uint64_t xy = x | y;
    // Set MSB if (x or y == 2) and (x or y == 1).
    uint64_t mask = (xy << 1) & xy;
    // Set MSB if (x == 2) and (y == 2).
    mask |= x & y;
    // The MSB of each 2-bit element is now set
    // iff the result is >= 3. Clear the LSBs.
    mask &= 0x5555555555555555 << 1;
    // Now turn the elements with MSB set into 3.
    mask |= mask >> 1;
    return x + y - mask;
}

```

Fig. 1. Broadword C code for addition modulo 3.

```

uint64_t sub_mod3_step2(uint64_t x, uint64_t y) {
    // y = 3 - y.
    y = 0xFFFFFFFFFFFFFFFF - y;
    // Now y > 0
    // Set MSB if x == 2.
    uint64_t mask = x;
    // Set MSB if (x == 2 and y >= 2) or (y == 3).
    mask |= ((x | y) << 1) & y;
    mask &= 0x5555555555555555 << 1;
    mask |= mask >> 1;
    return x + y - mask;
}

```

Fig. 2. Broadword C code for subtraction modulo 3.

the *generalized selfless algorithm* [30] (the algorithm has a small probability of failure, in which case we generate a new graph using the local seed). Moreover, since recently Goerdts and Falke have proved a threshold result analogous to that for k -XORSAT for linear systems on $\mathbf{F}_3$ [31],⁸ we can easily apply the technique of Section 4.2 and solve the linear system on $\mathbf{F}_3$ induced by the orientation to obtain a perfect hash function. However, we recall that solving the system on $\mathbf{F}_3$, rather than on $(\mathbf{F}_2)^2$, is essential, as discussed in Section 4.2; and it is even more essential that only vertices selected for the edge orientation are assigned a nonzero value in the solution. This property can be enforced, but the probability of obtaining a solvable system becomes experimentally about $2/3$, and does not appear to decrease with the system size. Once again, if we meet a failure we generate a new graph using the local seed.

7.2. Solving systems

Broadword programming. We can no longer simply perform a XOR to add two rows, as we are now working in $\mathbf{F}_3$, which requires more sophisticated algorithms. First, we can encode each element $\{0, 1, 2\}$ into 2 bits, thus fitting $w/2$ elements into a word. The scalar α can be only 1 or -1 , so we can treat the cases $x + y$ and $x - y$ separately.

In addition, we can start by simply adding x and y . When elements on both sides are smaller than 2, there's nothing to do: the result will be smaller than 3. When however at least one of the two is 2 and the other one is not 0, we need to subtract 3 from the result to bring it back to the canonical representation in $[0..3)$. Note that when the two sides are both 2 the result overflows its 2 bits ($10_2 + 10_2 = 100_2$), but since addition and subtraction modulo 2^w are associative we can imagine that the operation is performed independently on each 2-bit element, as long as the final result fits into 2 bits. Thus we need to compute a mask that is 3 wherever the results are at least 3, and then subtract it from $x + y$. The resulting code is shown in Fig. 1.

Subtraction is very similar. We begin by subtracting elementwise y from 3, which does not cause any carry since all the elements are strictly smaller than 3. The resulting elements are thus at least 1. We can now proceed to compute $x + y$ with the same case analysis as before, except now the right-hand elements are in $[1..3]$ so the conditions for the mask are slightly different. The resulting code is shown in Fig. 2.

Both addition and subtraction take just 10 arithmetic operations, and on modern 64-bit CPUs, they can process vectors of 32 elements at a time.

⁸ Technically, the proof in the paper is for $k > 15$, but the author claim that the result can be proved for $k \geq 3$ with the same techniques, and in practice, we never needed more than two attempts to generate a solvable system.

```

uint64_t prod_mod3_step2(uint64_t x, uint64_t y) {
    uint64_t z = x | y;
    uint64_t w = (z | z >> 1) & 0x5555555555555555;
    uint64_t t = x ^ y ^ w;
    return popcount(t & 0xAAAAAAAAAAAAAAAA) * 2
        + popcount(t & 0x5555555555555555);
}

```

Fig. 3. Broadword C code for scalar product modulo 3.

Finally, when performing back substitution the field is $\mathbf{F}_3$ but the matrix of solutions is a vector, so the product is just a scalar product. To compute it, we use the broadword algorithm shown in Fig. 3, which computes the scalar product of two $\mathbf{F}_3$ vectors represented as 64-bit words.⁹

The expression computing t takes care of placing in a given position a value equivalent to the product of the associated positions in x and y (this can be easily checked with a case-by-case analysis). We remark that in some cases we actually use 3 as equivalent to zero. At that point, the last lines compute the contribution of each product (`popcount()` returns the number of bits in a word that are set). Note that the result has still to be reduced modulo 3.

7.3. Data-structure representation

As explained in Section 4.2, we can gain a bit of space per key by creating a ranking structure on pairs of non zero bits. However, we can actually do better.

Eliminating the ranking structure. In the case of minimal perfect hashing, we can further speed up the structure and reduce space by getting rid of the ranking structure that is necessary to make minimal the perfect hashing computed by solving the random linear system on $\mathbf{F}_3$.

In the representation of static functions we discussed in Section 6.4, the number of variables associated to a bucket of size s is given by $\lceil cs \rceil$, where c is a suitable constant, and we store the prefix sums of such numbers to be able to compute the offset of each bucket.

We will use in this case a different approach: the number of variables associated with the bucket will be $\lceil c(p+s) \rceil - \lceil cp \rceil$, where p is the number of elements stored in previous buckets. The difference to $\lceil cs \rceil$ is at most one, but using our approach we can compute, given S and s , the number of variables associated with the bucket.

Thus, instead of storing the offset information of the bit array, or the prefix sums of the number of variables in each bucket, we will store for each bucket the number S of elements stored in previous buckets. The value can be used as a base for the ranking inside the bucket: in this way, the ranking structure is no longer necessary, reducing space and the number of memory accesses.

As explained in Section 4.2, we use two bits for each value, taking care of using the value 3, instead of 0, for vertices assigned to a hyperedge. As a result, ranking inside a bucket requires a linear scan counting the number of nonzero pairs in the values associated with a bucket. This task can be accomplished quickly by broadword programming using the expression `popcount((x | x >> 1) & 0x5555555555555555)`.

Of course, this approach makes it desirable that the bucket size is not too large, or the linear scan might become slow. In fact, this approach makes the lookup constant-time on average, but not in the worst case, as large buckets can theoretically happen (in practice, the event is so rare to be negligible).

8. Compressed static functions

Finally, we discuss the modification of the strategy for static functions in the compressed case.

8.1. Offline bucketing and the prefix-free code

Besides the usual fare of work described in Section 6.1, in this case, while we scan the input data we build a frequency table for the output values. Thus, at the end of the first pass we have on disk the usual sorted list of pairs $\langle s(x), f(x) \rangle$ and (in main memory) the frequencies p_σ , $\sigma \in \Sigma$. We are now interested in generating a prefix-free code for the output values, as described in Section 4.8.

Using the frequencies p_σ we can in principle generate an optimal prefix-free code for the distribution of the output values. Such a code, however, poses two problems: first, it is in general impossible to decode a codeword in constant time; second, for skewed distributions the longest generated codeword can be quite long.

The solution proposed in [9] is the following: a subset of most frequent elements of Σ is actually stored using an optimal length-limited code, together with an additional symbol $\perp$ accounting for the rest of the elements. Then, the non-frequent

⁹ We remark that the algorithm in the conference paper [1] is different. This version, which is slightly faster, was proposed by Djamel Belazzougui [39].

elements are stored explicitly after $\perp$ using a suitable table. Essentially, one trades the optimality for a small cost in space and faster decoding time.

After noticing that the decoding process had some effect on the lookup time, we decided to use a different approach based on canonical Huffman codes [27]. Canonical Huffman codes are based on the observation that among many equivalent Huffman decoding trees, there is one that is canonical: the one in which the depth of leaves from left to right is nondecreasing. The structure of such a tree (and of the associated code) can be described by two lists of integers of the same length: the first list enumerates the codeword lengths, in increasing order. The second list determines, for each length, the cardinality of the codewords with that length. Codewords of the same length are consecutive integers.

It is possible to decode a canonical code in time that is proportional to the *length of the lists*, rather than to the *length of the codewords* [40], as long as codewords are of length $O(w)$. This property shifts our problem from finding a code of *limited length* to finding a code *represented by lists of limited length*. In general terms, we might cast it as a combinatorial optimization problem as follows: given a length bound L , find the Huffman code described by lists of at most L elements which encodes the output of the function in the smallest number of bits.

In our case, we opt for a solution similar to that of [9], but based on canonical codes: given a bound L on the length of the lists, we cut the lists after L elements and use the distinguishing prefix of the first discarded codeword as an escape code; after the escape code, we write explicitly the output value to be encoded. In fact, we can harmlessly truncate the list at a length smaller than L if the accumulated entropy is, say, 99.9% of the empirical entropy, as the coding of the remaining symbols has no detectable impact on the overall size. With respect to the optimal solution described above, this approach has the advantage of reducing the size of the table mapping codewords to symbols when the function outputs a large number of infrequent values.

Finally, we remark that since we are always able to read from the bit array whole codewords, it is possible to arrange the decoding table so that the decoding loop for the canonical Huffman code reduces to a linear search in a short array.

If the distribution of values is known in advance, Huffman codes can in principle be replaced by a standard instantaneous code such as unary or Elias's γ .

8.2. Generating systems

We will follow the technique described in Section 4.8, using for all buckets the prefix-free code $c(\cdot)$ we just generated. If we focus on a single bucket the code we are using is no longer optimal, but this does not change the probability of solving systems, as the system size is dependent on the sum of the codewords representing values actually in the bucket, and does not depend on the optimality of the code. Note that the approach of having a local prefix-free optimal code per bucket would never work, as the space used to store the local codes would be unfeasible.

The authors of [9] advocate a special form for the hash functions $h_{(\cdot)}$:

$$h_i(x) = h'_i(x)L + q(x), \quad (2)$$

where L is the length of the longest codeword, and $h'_{(\cdot)} : U \rightarrow [m/L]$ and $q : U \rightarrow L$ are fully random hash functions (with some additional mild conditions—see [9]). By structuring the functions in this way, the systems associated with the locations of A (see Section 4.8) that are not congruent modulo L are independent: each system in isolation is a random linear system on $\mathbb{F}_2$, so one can directly apply the equivalence of their solvability with satisfiability of a k -XORSAT instance.

Inspired by a comment of Pagh [41], we use instead standard random hash functions mapping the keys into an array A of size $\delta_k \sum_{x \in X} |c(f(x))|$. We found that in practice this approach yields significantly shorter construction times, even if the resulting system is not fully random: the reason is that the k -XORSAT bounds we are using have a different asymptotic behavior than, say, acyclicity thresholds for random graphs. The asymptotic behavior starts to become typical at much larger sizes (in the acyclicity case, just a few dozens of vertices are sufficient). Since we are solving systems bucket by bucket, but we are using a global code, some buckets happen to have codewords with large length L , in particular in the case of skewed distributions. For these buckets, using (2) scales the size of the systems and the number of variables down by a factor of L , bringing them out of the “good” region (see also Fig. 4).

9. Experimental results

In this section, we present the results of our experiments, which were performed on an Intel® Core™i7-7770 CPU @3.60 GHz (Kaby Lake), with 64 GiB of RAM, Linux 4.17.19 and Java 12. For the C code, we used the GNU C compiler 8.1.1.

The number of bits per key is measured on the serialized data structure and contains some Java serialization overhead (below 1%). Time is measured by wall clock. For all the lookup timings we report, the relative standard deviation is very low (below 5%) and due mostly to Java garbage collection. Construction times (which include reading the input, generating the data structure and serializing it) have a bit more variability due to I/O, but we take care of dropping all disk caches before running each test. We also fix the CPU clock to avoid variations due to throttling from the Turbo Boost controller.

We report timings in four flavors:

- Java timings without any options enabled; in this setting, the memory access does not use large pages and the TLB (Translation Lookaside Buffer) operations are quite slow.

Table 1

Actual ranges and empirical entropy of our synthetic and real-world datasets.

Dataset	Size	Geometric	Zipfian	Uniform	Indegree
Hosts	11.26M	[0..22], 2.0	[0..973412], 2.36	[0..63], 6.0	[1..174433], 4.22
URLs	1.07B	[0..32], 2.0	[0..998365], 2.36	[0..63], 6.0	[1..20252239], 5.11

- Java timings with the option `-XX:+UseTransparentHugePages`, which enables (confusingly) transparent *large* (2 MiB) pages.
- C timings obtained using a C implementation which contains only the lookup code—the data structure is derived directly from the one generated by the Java code. Also, in this case, we do not use large pages.
- C timings using large pages.

The impact of the inner workings of the TLB on data structures of this kind is very high, as most of the time is spent performing memory accesses.

We performed experiments using two datasets derived from the eu-2015 crawls gathered by BUBiNG [42]. The smaller dataset is the list of hosts (11 264 052 keys, ≈ 22 B/key), while the larger dataset is the list of pages (1 070 557 254 keys, ≈ 80 B/key). The crawl data is publicly available at the LAW website.¹⁰ Note that the URL dataset is much bigger (per key): an additional ≈ 500 ns per key are necessary just to read the input.

9.1. Choosing a bucket size

As a first step, we analyze how performance varies with the bucket size. In Fig. 4 we show how the number of bits per element, construction time, lookup time and fraction of unsolvable (or unorientable, in the MPHf case) systems vary with the bucket size for static functions with $k = 3$ and for minimal perfect hash functions. Note that in the case of minimal perfect hash functions we show the actual number of bits per key. In the case of general static functions, we report the ratio with respect to the information-theoretical lower bound as a percentage. For these experiments we considered only the case of Java with large pages enabled.

As the bucket size gets larger, the first row of Fig. 4 shows that the number of bits per key slightly decreases (as the impact of the offset structure is better amortized); at the same time:

- construction time (second row) increases because the Gaussian elimination process is superlinear (very sharply when the bucket size exceeds 1500);
- in the case of minimal perfect hash functions, larger buckets cause the rank function to do more work linearly in the bucket size, and as a consequence lookup time increases significantly (third row);
- in the case of static functions and URLs, buckets larger than 2000 yield a slightly improved lookup time as the offset array becomes small enough to fit in the L3 cache;
- the asymptotic behavior of the k -XORSAT thresholds starts to become evident for systems with at least a few thousand of equations, both for solvability and for orientability (fourth row);
- the fraction of unsolvable systems on $\mathbf{F}_3$ does not go below 35% in the displayed range because we enforce solutions that are nonzero only on the vertices used for orientation (if this property is not required, systems on $\mathbf{F}_3$ become solvable asymptotically similarly to what happens on $\mathbf{F}_2$).

All in all, we suggest a bucket size $B = 1500$ for static functions and minimal perfect hash functions. Since the computational burden is significantly heavier for compressed static functions, we suggest $B = 1000$ in that case. The rest of the experiments in the paper have been performed using these bucket sizes.

9.2. Experimental comparison

The first set of experiments compares (non-compressed) static functions and minimal perfect hash functions with previous approaches reaching the same number of bits per key. In the first case, we store functions mapping each string to its ordinal position in the dataset.

In the second set of experiments, we examine our compressed implementation and compare it with the non-compressed case (as there are no previous implementations in the literature). For each key set, we generate three synthetic lists of values using different distributions: a geometric distribution with probability $p = 1/2$; a Zipfian (i.e., finite power-law) distribution with 10^6 values and exponent 2; a discrete uniform distribution with 64 values. Moreover, we consider the functions from each URL (host) to its indegree in the web (host, respectively) graph. The geometric and uniform distributions are the most skewed and the less skewed distribution with exactly matching optimal codes, and the Zipfian distribution sits somewhere

¹⁰ <http://law.di.unimi.it/>.

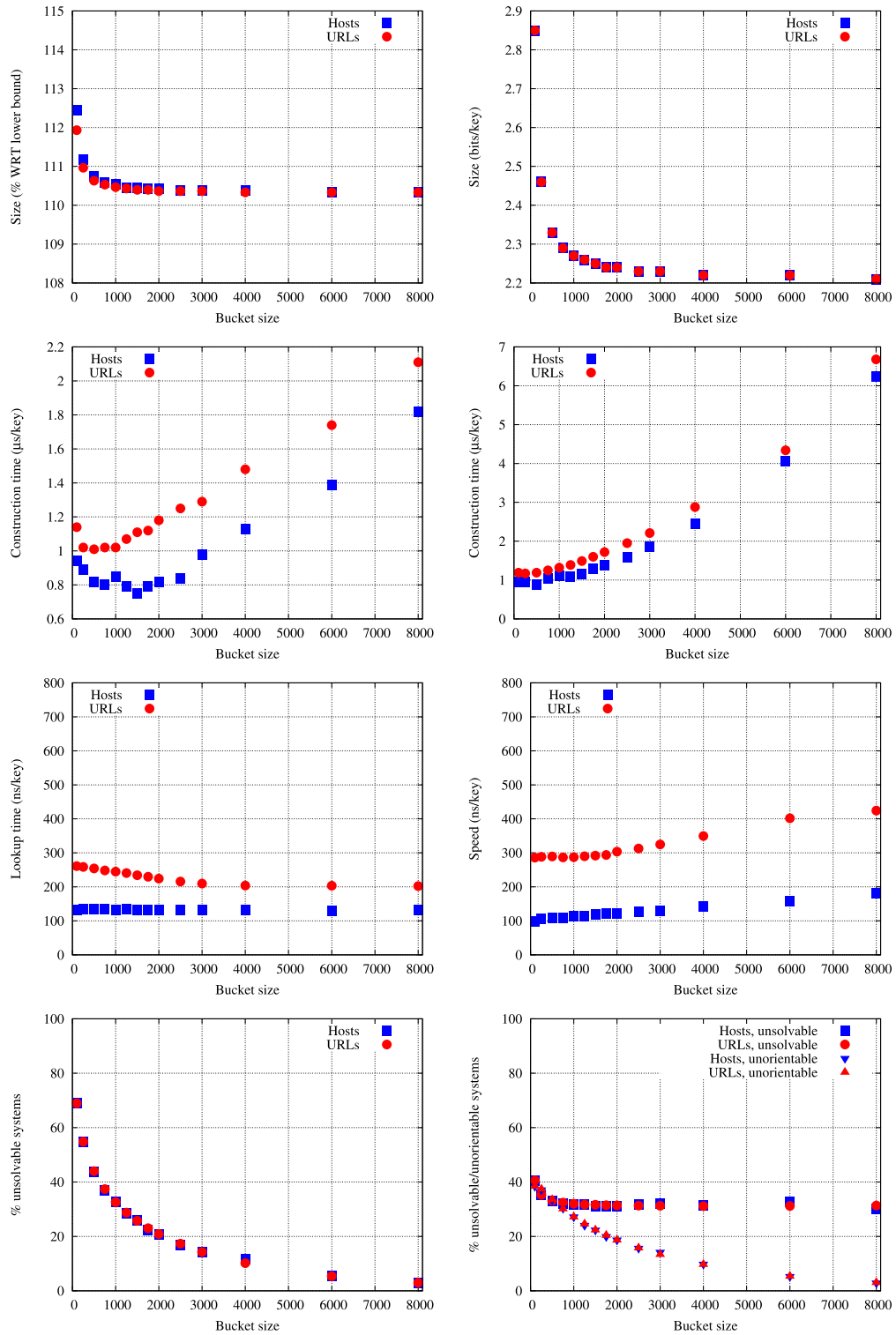


Fig. 4. Size, construction, lookup time and unsolvability/unorientability for static functions with $k=3$ (left) and for minimal perfect hash functions (right) for the eu-2015 hosts and URL datasets as the bucket size varies.

Table 2Per-key construction and evaluation time for static functions, $k = 3$. ADR is from [26].

	eu-2015 (hosts)	eu-2015 (URLs)	ADR
Size	110%	110%	110%
Constr. ($\mu\text{s}/\text{key}$)	0.76	1.25	270
Lookup (Java, ns/key)	145	317	?
Lookup (Java TLP, ns/key)	132	234	?
Lookup (C, ns/key)	120	199	?
Lookup (C TLP, ns/key)	108	144	?

Table 3Per-key construction and evaluation time for static functions, $k = 4$. ADR is from [26].

	eu-2015 (hosts)	eu-2015 (URLs)	ADR
Size	103%	103%	103%
Constr. ($\mu\text{s}/\text{key}$)	1.29	1.68	≈ 2000
Lookup (Java, ns/key)	160	342	?
Lookup (Java TLP, ns/key)	146	252	?
Lookup (C, ns/key)	125	211	?
Lookup (C TLP, ns/key)	113	154	?

Table 4A comparison of per-key construction and evaluation time for minimal perfect hash functions. CHD ($\lambda = 3$) is from [23].

	eu-2015 (hosts)		eu-2015 (URLs)	
	MPHF	CHD	MPHF	CHD
Size	2.24	2.26	2.24	2.26
Constr. ($\mu\text{s}/\text{key}$)	1.18	0.67	1.52	3.53
Lookup (Java, ns/key)	119	—	321	—
Lookup (Java TLP, ns/key)	116	—	293	—
Lookup (C, ns/key)	81	258	230	657
Lookup (C TLP, ns/key)	77	242	206	586

Table 5Increase in construction time for $k = 3$ using just pre-peeling (P), broadword computation (B), lazy Gaussian elimination (G) or a combination.

BGP	BG	GP	G	BP	B	P	None
+0%	+13%	+57%	+98%	+296%	+701%	+2218%	+5490%

in the middle. Note that in principle the geometric distribution has unlimited range, but in practice, for n keys the largest generated value is $O(\log n)$ with high probability. Table 1 reports the empirical value range and empirical entropy of the eight resulting combinations.

In Table 2 and 3 we compare the lookup and construction time for static functions with respect to the data reported in [26] for the same space usage¹¹ (i.e., 110%). In Table 4, instead, we compare our minimal perfect hash functions to the C code for the CHD technique made available by the authors [43] when $\lambda = 3$, in which case the number of bits per key is (2.26) almost identical to ours. We remark that in the case of CHD for the larger dataset we had to use different hardware, as the memory available (64 GB) was not sufficient to complete the construction, in spite of the final result being just 3 GB.

Note that the timings for our C implementation can be compared directly with the timings for CHD and for ADR, which are based on C implementations, too.

In the case of static functions, we can build data structures about two hundred times faster than what was previously possible [26] (the data displayed is on a dataset with 10^7 elements; lookup time was not reported). To give our reader an idea of the contribution of each technique we use, Table 5 shows the increase in construction time using any combination of the peeling phase (which is technically not necessary—we could just solve the system), broadword computation instead of a standard sparse system representation, and lazy instead of standard Gaussian elimination. The combination of our techniques brings a *fifty-fold* increase in speed (our basic speed is already fourfold that of [26], likely because our hardware is more recent).

¹¹ Unfortunately, the authors report only a graph with timings for $k = 5$, and for different values of their splitting parameters providing a space usage ranging from 107% to 162%.

Table 6

Comparison between the non-compressed implementations (Std) and compressed implementations (Comp).

Host dataset	Geometric		Zipfian		Uniform		Indegree	
	Std	Comp	Std	Comp	Std	Comp	Std	Comp
<i>k</i> = 3								
Size (b/key)	5.56	2.29	22.10	2.78	6.66	6.69	19.89	4.80
Constr. (μs/key)	0.86	1.40	0.95	1.74	0.81	7.21	0.95	3.74
Lookup (Java, ns/key)	113	103	144	105	119	122	143	128
Lookup (Java TLP, ns/key)	91	97	132	97	104	110	130	107
Lookup (C, ns/key)	65	64	118	63	76	78	117	77
Lookup (C TLP, ns/key)	59	63	108	62	71	72	107	70
<i>k</i> = 4								
Size (b/key)	5.20	2.15	20.68	2.61	6.24	6.26	18.62	4.50
Constr. (μs/key)	1.29	2.42	1.33	3.34	1.34	25.03	1.42	10.85
Lookup (Java, ns/key)	123	109	158	109	127	130	157	123
Lookup (Java TLP, ns/key)	93	106	147	105	110	116	147	111
Lookup (C, ns/key)	65	66	123	66	77	77	122	77
Lookup (C TLP, ns/key)	54	65	113	64	70	69	112	70
URL dataset	Geometric		Zipfian		Uniform		Indegree	
	Std	Comp	Std	Comp	Std	Comp	Std	Comp
<i>k</i> = 3								
Size (b/key)	6.66	2.30	22.10	2.76	6.66	6.69	27.61	5.77
Constr. (μs/key)	1.23	1.82	1.25	2.15	1.24	7.55	1.29	5.77
Lookup (Java, ns/key)	268	290	305	293	268	293	313	313
Lookup (Java TLP, ns/key)	219	266	225	265	219	249	233	274
Lookup (C, ns/key)	198	204	211	204	197	207	203	227
Lookup (C TLP, ns/key)	157	185	162	184	157	168	152	192
<i>k</i> = 4								
Size (b/key)	6.23	2.16	20.68	2.59	6.23	6.26	25.84	5.40
Constr. (μs/key)	1.76	2.91	1.78	3.75	1.76	24.49	1.81	16.82
Lookup (Java, ns/key)	289	297	330	297	287	298	339	316
Lookup (Java TLP, ns/key)	241	272	251	270	243	257	255	278
Lookup (C, ns/key)	204	208	223	209	204	211	214	231
Lookup (C TLP, ns/key)	164	189	172	188	163	174	162	197

In the case of MPHFs, we have extremely competitive lookup speed (almost twice that of CHD) and much better scalability. At a small size, performing the construction entirely in main memory, as CHD does, is an advantage, but as soon as the dataset gets large our approach scales much better.

The gap in speed is quite stable with respect to the key size: testing the same structures with very short (less than 8 bytes) random keys provides of course faster lookup, but the ratio between the lookup times remain the same.

Finally, one must consider that CHD, at the price of much greater construction time, can further decrease its space usage, but just a 9% decrease in space increases construction time by an order of magnitude, which makes the tradeoff unattractive for large datasets.

In Table 3 we report timings for the case $k = 4$ (the construction time for [26] has been extrapolated, as the authors do not provide timings for this case). Additional space required now is just $\approx 3\%$ (as opposed to $\approx 10\%$ when $k = 3$). The main drawbacks are the slower construction time (as the system becomes denser) and the slower lookup time (as more memory has to be accessed). Larger values of k are not interesting as the marginal gain in space becomes negligible.

We remark that with respect to our previous implementations based on the MWHC approach, we increase construction time by $\approx 50\%$ (SF) and $\approx 100\%$ (MPHF), at the same time decreasing lookup time (experiments not reported here).

9.3. The compressed case

In Table 6 we report the results on our eight combinations of datasets and output values, coupled with similar results for the non-compressed implementations. If the distribution is skewed and the entropy is small, we obtain a significant compression, very close to $\delta_k \mathcal{H}_0$ bits per key, as expected.

The compression is actually more impressive in the case of a distribution with a long tail, as in that case for large datasets a few very large values occur, which forces the standard implementation to choose a large number of bits per value. In the case of the geometric distribution, this might happen, too, but with a very small probability.

In the uniform case, construction time increases significantly, and of course, we have no space gain. Nonetheless, since we are able to use a number of bits per key very close to the best possible, we do not increase the space usage. Lookup time is slightly slower due to decoding, but by using a flat binary code it would be equivalent to the standard case.

Lookup times for non-uniform distributions are (sometimes significantly) smaller. This is due to the reduced stress on the cache and on the Translation Lookahead Buffer, which limits the speed at which the CPU can access memory. With respect

Table 7Data for ratio vertices/hyperedges $\approx \gamma_3$, using peeling to triangulate and solve linear systems.

Host dataset	$k = 3$			
	Geometric	Zipfian	Uniform	Indegree
Size (b/key)	2.55	3.09	7.47	5.35
Constr. (μ s/key)	0.67	0.72	0.85	0.78
Lookup (Java, ns/key)	101	101	121	124
Lookup (Java TLP, ns/key)	98	98	116	115
Lookup (C, ns/key)	77	83	94	105
Lookup (C TLP, ns/key)	63	66	79	74
URL dataset	$k = 3$			
	Geometric	Zipfian	Uniform	Indegree
Size (b/key)	2.56	3.08	7.46	6.43
Constr. (μ s/key)	1.06	1.06	1.26	1.26
Lookup (Java, ns/key)	289	290	293	314
Lookup (Java TLP, ns/key)	262	262	245	270
Lookup (C, ns/key)	206	206	208	229
Lookup (C TLP, ns/key)	185	183	168	192

to the case $k = 3$, the case $k = 4$ sports slightly increased access time, a few-percent reduction in space (as expected) and much longer construction time, due to the increased density of the linear systems. Decoding the canonical Huffman code has a very marginal impact, as it requires ≈ 10 ns or less (≈ 2 ns in the uniform case, as the decoding table contains just one entry).

9.4. Switching to peeling

Since we try to peel the system before solving it by lazy Gaussian elimination, for $k = 3$ if we set the ratio vertices/hyperedges to a value greater than $\gamma_3 \approx 1.23$ we can almost surely peel (and thus triangulate and solve) the whole system in linear time as in the MWHC case. In practice, with a 12% increase in space, we can *guarantee* that the construction time will be at most multiplied by the entropy. In fact, as we show in Table 7, the increase is usually much less, as the peeling process is responsible for a small fraction of the overall construction time. Maybe surprisingly, because of the reduced memory footprint the construction time for the geometric case is *faster* than the standard construction. Note that γ_3 is *minimum* among the γ_k , so there is no sense in trying this approach for $k \neq 3$.

10. Further applications

Static functions are a basic building block of *monotone* minimal perfect hash functions [5], data structures for weak prefix search [7], and so on. Replacing the common MWHC implementation of these building blocks with our improved construction will automatically decrease the space used and the lookup time in these data structures.

We remark that an interesting application of static functions is the almost optimal storage of *static approximate dictionaries*. By encoding the function from a key to a b -bit signature generated by a random hash function, one can answer to the question “ $x \in X$?” in constant time, with false positive rate 2^{-b} , using (when $k = 4$) just $1.03nb$ bits; the lower bound is nb [44].

11. Conclusions

We have presented new practical data structures for static functions and minimal perfect hash functions. Both scale to billions of keys (and beyond) and both improve significantly lookup speed with respect to previous constructions. In particular, we can build static functions based on Gaussian elimination two orders of magnitude faster than previous approaches, thanks to a combination of broadword programming and a new, parameterless lazy approach to the solution of sparse linear systems. We expect that these structures will eventually replace the venerable MWHC approach as a scalable method with high-performance lookup.

We presented also an implementation of the theoretical construction proposed in [9] for compressed static functions. By carefully tuning each part of the construction process, we have been able in most cases to contain the increase in construction time in the case of low entropy. Note that to circumvent the high-entropy case one can measure at construction time the entropy of the output values and just switch to the construction based on linear-time peeling.

We remark that one of the great advantages of the theoretical approach described in [9] is that the number of memory accesses to compute a value is identical to that of the non-compressed case we are comparing with: indeed, in our tests we found a minor lookup slowdown in the uniform case, and a speedup in some skewed cases, in spite of the time that is necessary for decoding, as the smaller memory footprint reduces the out-of-cache accesses and the stress on the Translation Lookaside Buffer.

In any case, the implementations we distribute within Sux4J uses multicore parallelization to further decrease the construction time (the single-thread evaluation we used in this paper provides more consistent results across different techniques). It is also trivial to build efficiently our data structures on distributed frameworks such as MapReduce [45]. In fact, it is not even necessary to build the structures in memory: as buckets are accumulated, the associated bit arrays and offset information can be written directly on disk, and the data structure can be accessed, for example, by memory mapping. Thus, one can build and use structures larger than the memory available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] M. Genuzio, G. Ottaviano, S. Vigna, Fast scalable construction of (minimal perfect hash) functions, in: A.V. Goldberg, A.S. Kulikov (Eds.), *Experimental Algorithms: 15th International Symposium, SEA 2016, St. Petersburg, Russia, June 5–8, 2016, Proceedings*, in: *Lecture Notes in Computer Science*, vol. 9685, Springer, 2016, pp. 339–352.
- [2] M. Genuzio, S. Vigna, Engineering compressed static functions, in: *2018 Data Compression Conference, IEEE*, 2018, pp. 52–61.
- [3] D. Belazzougui, G. Navarro, Alphabet-independent compressed text indexing, *ACM Trans. Algorithms* 10 (4) (2014), 23:1–23:19.
- [4] D. Belazzougui, P. Boldi, R. Pagh, S. Vigna, Monotone minimal perfect hashing: searching a sorted table with $O(1)$ accesses, in: *Proceedings of the 20th Annual ACM-SIAM Symposium on Discrete Mathematics, SODA, ACM Press, New York, 2009*, pp. 785–794.
- [5] D. Belazzougui, P. Boldi, R. Pagh, S. Vigna, Theory and practice of monotone minimal perfect hashing, *ACM J. Exp. Algorithmics* 16 (3) (2011) 3.2:1–3.2:26.
- [6] D. Belazzougui, R. Venturini, Compressed static functions with applications, in: *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2013, SIAM, 2013*, pp. 229–240.
- [7] D. Belazzougui, P. Boldi, R. Pagh, S. Vigna, Fast prefix search in little space, with applications, in: M. de Berg, U. Meyer (Eds.), *Algorithms - ESA 2010, Proceedings of the 18th Annual European Symposium, Part I, Liverpool, UK, September 6–8, 2010*, in: *Lecture Notes in Computer Science*, vol. 6346, Springer, 2010, pp. 427–438.
- [8] E. Porat, An optimal Bloom filter replacement based on matrix solving, in: *Computer Science – Theory and Applications, Fourth International Computer Science Symposium in Russia*, in: *Lecture Notes in Computer Science*, vol. 5675, Springer, 2009, pp. 263–273.
- [9] J.B. Hreinnsson, M. Krøyer, R. Pagh, Storing a compressed function with constant time access, in: *Algorithms - ESA 2009, Proceedings of the 17th Annual European Symposium, Copenhagen, Denmark, September 7–9, 2009*, in: *Lecture Notes in Computer Science*, vol. 5757, Springer, 2009, pp. 730–741.
- [10] M. Dietzfelbinger, R. Pagh, Succinct data structures for retrieval and approximate membership (extended abstract), in: L. Aceto, I. Damgård, L.A. Goldberg, M.M. Halldórsson, A. Ingólfssdóttir, I. Walukiewicz (Eds.), *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008, Proceedings, Part I: Track A: Algorithms, Automata, Complexity, and Games*, in: *Lecture Notes in Computer Science*, vol. 5125, Springer, 2008, pp. 385–396.
- [11] M. Rink, Thresholds for matchings in random bipartite graphs with applications to hashing-based data structures, Ph.D. thesis, Technische Universität Ilmenau, 2015.
- [12] D.E. Knuth, *The Art of Computer Programming. Pre-Fascicle 1A. Draft of Section 7.1.3: Bitwise Tricks and Techniques*, Addison-Wesley, 2007.
- [13] Sux4J, Succinct data structure for Java, <http://sux4j.di.unimi.it/>.
- [14] O. Dubois, J. Mandler, The 3-XORSAT threshold, *C. R. Math.* 335 (11) (2002) 963–966.
- [15] B. Pittel, G.B. Sorkin, The satisfiability threshold for k -XORSAT, *Comb. Probab. Comput.* 2 (2016) 236–268.
- [16] M.L. Fredman, J. Komlós, E. Szemerédi, Storing a sparse table with $O(1)$ worst case access time, *J. Assoc. Comput. Mach.* 31 (3) (1984) 538–544.
- [17] T. Hagerup, T. Tholey, Efficient minimal perfect hashing in nearly minimal space, in: A. Ferreira, H. Reichel (Eds.), *STACS 2001, Proceedings of the 18th Annual Symposium on Theoretical Aspects of Computer Science, Dresden, Germany, February 15–17, 2001*, pp. 317–326.
- [18] B.S. Majewski, N.C. Wormald, G. Havas, Z.J. Czech, A family of perfect hashing methods, *Comput. J.* 39 (6) (1996) 547–554.
- [19] B. Chazelle, J. Kilian, R. Rubinfeld, A. Tal, The Bloomier filter: an efficient data structure for static support lookup tables, in: J.I. Munro (Ed.), *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2004, SIAM, 2004*, pp. 30–39.
- [20] S. Vigna, Broadword implementation of rank/select queries, in: C.C. McGeoch (Ed.), *Experimental Algorithms: 7th International Workshop, WEA 2008*, in: *Lecture Notes in Computer Science*, vol. 5038, Springer-Verlag, 2008, pp. 154–168.
- [21] F.C. Botelho, R. Pagh, N. Ziviani, Practical perfect hashing in nearly optimal space, *Inf. Sci.* 38 (1) (2013) 108–131.
- [22] D. Belazzougui, P. Boldi, G. Ottaviano, R. Venturini, S. Vigna, Cache-oblivious peeling of random hypergraphs, in: *2014 Data Compression Conference, DCC 2014, IEEE*, 2014, pp. 352–361.
- [23] D. Belazzougui, F.C. Botelho, M. Dietzfelbinger, Hash, displace, and compress, in: A. Fiat, P. Sanders (Eds.), *Algorithms - ESA 2009, 17th Annual European Symposium, Copenhagen, Denmark, September 7–9, 2009. Proceedings*, 2009, pp. 682–693.
- [24] I. Müller, P. Sanders, R. Schulze, W. Zhou, Retrieval and perfect hashing using fingerprinting, in: J. Gudmundsson, J. Katajainen (Eds.), *SEA 2014: Experimental Algorithms, Springer International Publishing*, 2014, pp. 138–149.
- [25] N.J. Calkin, Dependent sets of constant weight binary vectors, *Comb. Probab. Comput.* 6 (3) (1997) 263–271.
- [26] M. Aumüller, M. Dietzfelbinger, M. Rink, Experimental variations of a theoretically good retrieval data structure, in: A. Fiat, P. Sanders (Eds.), *Algorithms - ESA 2009, 17th Annual European Symposium, Copenhagen, Denmark, September 7–9, 2009. Proceedings*, in: *Lecture Notes in Computer Science*, vol. 5757, Springer, 2009, pp. 742–751.
- [27] E.S. Schwartz, B. Kallick, Generating a canonical prefix encoding, *Commun. ACM* 7 (3) (1964) 166–169.
- [28] N. Fountoulakis, K. Panagiotou, Sharp load thresholds for cuckoo hashing, *Random Struct. Algorithms* 41 (3) (2012) 306–333.
- [29] A.M. Frieze, P. Melsted, Maximum matchings in random bipartite graphs and the space utilization of cuckoo hash tables, *Random Struct. Algorithms* 41 (3) (2012) 334–364.
- [30] M. Dietzfelbinger, A. Goerdt, M. Mitzenmacher, A. Montanari, R. Pagh, M. Rink, Tight thresholds for cuckoo hashing via XORSAT, in: S. Abramsky, C. Gavioille, C. Kirchner, F. Meyer auf der Heide, P. Spirakis (Eds.), *Automata, Languages and Programming*, in: *Lecture Notes in Computer Science*, vol. 6198, Springer, Berlin Heidelberg, 2010, pp. 213–225.
- [31] A. Goerdt, L. Falke, Satisfiability thresholds beyond k -XORSAT, in: E.A. Hirsch, J. Karhumäki, A. Lepistö, M. Prilutskii (Eds.), *Computer Science – Theory and Applications: 7th International Computer Science Symposium in Russia, CSR 2012. Proceedings*, Springer, Berlin Heidelberg, 2012, pp. 148–159.
- [32] B. Jenkins, <http://burtleburtle.net/bob/hash/spooky.html>, 2018.

- [33] L. Devroye, Non-Uniform Random Variate Generation, Springer-Verlag, 1986.
- [34] D.E. Knuth, The Metafont Book, Addison-Wesley, 1986.
- [35] A. Knoblauch, Closed-form expressions for the moments of the binomial probability distribution, *SIAM J. Appl. Math.* 69 (1) (2008) 197–204.
- [36] J.L. Bentley, A.C.-C. Yao, An almost optimal algorithm for unbounded searching, *Inf. Process. Lett.* 5 (3) (1976) 82–87.
- [37] A.M. Odlyzko, Discrete logarithms in finite fields and their cryptographic significance, in: T. Beth, N. Cot, I. Ingemarsson (Eds.), *Advances in Cryptology*, Springer, Berlin Heidelberg, 1985, pp. 224–314.
- [38] B.A. LaMacchia, A.M. Odlyzko, Solving large sparse linear systems over finite fields, in: A.J. Menezes, S.A. Vanstone (Eds.), *Advances in Cryptology—CRYPTO’90*, Springer, Berlin Heidelberg, 1991, pp. 109–133.
- [39] D. Belazzougui, 2018, Personal electronic communication.
- [40] D.S. Hirschberg, D.A. Lelewer, Efficient decoding of prefix codes, *Commun. ACM* 33 (4) (1990) 449–459.
- [41] R. Pagh, 2016, Personal electronic communication.
- [42] P. Boldi, A. Marino, M. Santini, S. Vigna, BUBiNG: massive crawling for the masses, *ACM Trans. Web* 12 (2) (2019), 12:1–12:26.
- [43] D. de Castro Reis, F.C. Botelho, CMPH, <http://cmph.sourceforge.net/>, 2018.
- [44] L. Carter, R. Floyd, J. Gill, G. Markowsky, M. Wegman, Exact and approximate membership testers, in: *Proceedings of Symposium on Theory of Computation*, STOC ’78, ACM Press, 1978, pp. 59–65.
- [45] J. Dean, S. Ghemawat, MapReduce: simplified data processing on large clusters, *Commun. ACM* 51 (1) (2008) 107–113.